

LEVEL 10

Chapter 11: Observability & Operations

System Design Master Roadmap — 9 Years of Experience Edition

Compiled August 2026



Chapter 11: Observability & Operations

← [Chapter 10: Security](#) · [Index](#) · Next → [Chapter 12: Cloud System Design \(AWS\)](#)

You run Prometheus, Grafana, and OpenTelemetry in production already — this chapter's job is turning that into the vocabulary an interviewer expects (SLI/SLO/SLA, RPO/RTO) and showing where observability fits structurally in an HLD answer, not just as a bullet point at the end.

11.1 The Three Pillars: Logs, Metrics, Traces

Pillar	Answers	Granularity	Example tool
Logs	"What exactly happened, in detail, for this one event?"	Very high (full event detail)	ELK/EFK stack, CloudWatch Logs
Metrics	"How is the system behaving in aggregate, over time?"	Low (numbers over time — counters, gauges, histograms)	Prometheus + Grafana
Traces	"Where did time go, across services, for this one request?"	Per-request, cross-service	OpenTelemetry → Jaeger/Tempo

Why you need all three, not just one: metrics tell you *that* p99 latency spiked at 3:14 PM; traces tell you *which service in the call chain* caused it; logs tell you *exactly why* (the specific error, the specific query, the specific input). Losing any one of the three leaves a real gap — metrics alone can't explain root cause, traces alone don't give you the aggregate "is this normal" context, logs alone don't scale to "show me the trend over the last 30 days" without becoming a de facto (bad) metrics system.

Correlation ID: a single ID (often the trace ID itself) attached to a request at the edge and propagated through every service, log line, and span it touches — this is the thread that lets you jump from "user reports order #4521 failed" to the exact trace, and from the trace to the exact log lines across every service involved, in seconds instead of grep-ing five services' logs by timestamp and guessing.

11.2 SLI, SLO, SLA — The Distinction That Gets Asked Constantly

Term	Definition	Example
	The actual measured metric	

Term	Definition	Example
SLI (Service Level Indicator)		"99.95% of requests in the last 30 days completed in under 300ms"
SLO (Service Level Objective)	Your internal target for that metric	"We aim for 99.9% of requests under 300ms"
SLA (Service Level Agreement)	An external, often contractual, promise to customers — usually looser than your internal SLO, with real consequences (credits, penalties) if missed	"We guarantee 99.5% uptime, or you receive service credits"

Why the SLO is deliberately stricter than the SLA: you want your internal alerting threshold to fire *before* you're at risk of breaching a customer-facing contractual promise — the gap between SLO and SLA is your safety margin. A common, genuinely useful pattern is the **error budget**: if your SLO is 99.9% availability, you have a 0.1% "budget" of allowed failure per period — spend it on controlled risk (faster feature releases, planned maintenance) when you have budget left, and freeze risky changes when you've burned through it. This ties observability directly to engineering process, which is a strong thing to mention at a senior/staff level.

Interview question: "How would you define and monitor an SLO for a payment API?"

Ideal senior answer: "I'd pick an SLI that reflects what actually matters to users — say, 'percentage of payment requests completing successfully within 2 seconds,' measured from the API gateway. Set an SLO, say 99.95% over a rolling 30-day window, informed by what the business actually needs, not an arbitrary round number. Alert on **burn rate** — how fast you're consuming your error budget — rather than only on the raw SLO threshold, because a fast burn rate (say, 10% of a month's error budget in one hour) needs a page right now, while a slow, steady burn might just need a ticket for next sprint. I'd expose this via a Grafana dashboard backed by Prometheus recording rules over the raw request metrics, which is close to what I already run today."

11.3 Alerting and Incident Response

Alerting principles: alert on **symptoms users would notice** (elevated error rate, high latency) rather than every possible internal cause (don't page someone at 3 AM because CPU hit 80% if nothing is actually degraded for users) — this avoids alert fatigue, which is the single biggest reason good alerting systems stop being trusted and get ignored. Route alerts by severity: page for user-facing SLO-threatening issues, ticket/Slack for things that need attention but not immediate action.

Incident response — a standard shape worth knowing the vocabulary for: **detect** (alert fires) → **triage/mitigate** (stop the bleeding — rollback, failover, feature-flag off — before root-causing) → **root cause** (once stable) → **postmortem** (blameless, written, with concrete action items) → **follow-through** (actually do the action items, or they're theater). Interviewers occasionally probe "how would you respond to a 2am page for this system," and the strong answer follows exactly this shape, prioritizing mitigation speed over root-cause perfectionism in the first few minutes.

11.4 Disaster Recovery: RPO, RTO, Backup/Restore, Multi-AZ, Multi-Region

RPO (Recovery Point Objective): how much data you can afford to lose, measured in time — "RPO of 5 minutes" means your backup/replication strategy must ensure you never lose more than 5 minutes of data in a disaster. Driven by backup/replication *frequency*.

RTO (Recovery Time Objective): how long you can afford to be down before service is restored — "RTO of 1 hour" means your failover process (automated or manual) must bring the system back within an hour. Driven by how automated/tested your failover process is.

Strategy	RPO	RTO	Cost
Nightly backups, manual restore	Up to 24 hours	Hours (manual process)	Low
Continuous replication, manual failover	Seconds to minutes	Tens of minutes (human has to act)	Medium
Multi-AZ with automated failover	Near-zero (synchronous replication)	Minutes (automated)	Higher
Active-active multi-region	Near-zero	Seconds to zero (traffic just shifts)	Highest — real engineering and cost overhead

Multi-AZ: running redundant infrastructure across multiple **Availability Zones** within one AWS region (physically separate data centers, same region, low-latency links between them) — protects against a single data center failure (power, cooling, hardware) without the complexity of a full multi-region setup. This is the default, sane baseline for any production system with a real availability requirement — genuinely low-hanging fruit given AWS makes it close to a checkbox for RDS/EKS.

Multi-region: running redundant infrastructure across entirely separate geographic regions — protects against a whole-region outage (rare but real — AWS regions have gone down) and reduces latency for geographically distributed users. Genuinely hard: data replication across regions is either eventually consistent (accept some lag) or pays significant latency for synchronous cross-region writes (physics — a round trip between Mumbai and Virginia is ~250ms minimum). Reach for it when the business impact of a full-region outage, or the latency cost of serving distant users from one region, genuinely justifies the operational cost — not by default.

Interview question: "What's your RPO/RTO strategy for a payments database?"

Ideal senior answer: "Given financial data, I'd target a very tight RPO — near-zero — via synchronous or near-synchronous multi-AZ replication (e.g., RDS Multi-AZ, which replicates synchronously to a standby), so a single AZ failure loses effectively no committed transactions. RTO I'd target in the low minutes via automated failover, which RDS Multi-AZ also gives you largely for free. I'd treat full multi-region active-active as a separate, larger conversation — it protects against a rarer failure mode (whole-region outage) at real complexity and consistency cost, and I'd want to understand the actual business risk tolerance and regulatory requirements before committing to it, rather than assume it's needed by default."

Chapter 11 Interview Drill

1. Explain SLI vs SLO vs SLA with one example each, and why the SLO should be stricter than the SLA.
2. What is an "error budget," and how does it change engineering decisions day to day?
3. Why alert on symptoms (latency, error rate) instead of every internal cause (CPU, memory)?
4. Define RPO and RTO precisely, and give an infrastructure choice that improves each.
5. When is multi-region justified, and when is multi-AZ enough?

Next → [Chapter 12: Cloud System Design \(AWS\)](#) — every core AWS service mapped to the problem it solves, when to use it, and when not to.